



A Q-Learning based NSGA-II for dynamic flexible job shop scheduling with limited transportation resources

Rensheng Chen^a, Bin Wu^{a,*}, Hua Wang^b, Huagang Tong^a, Feiyi Yan^a

^a School of Economics and Management, Nanjing Tech University, Nanjing, 211816, China

^b School of Mechanical and Power Engineering, Nanjing Tech University, Nanjing, 211816, China

ARTICLE INFO

Keywords:

Flexible job shop scheduling problem

Limited transportation resources

Dynamic scheduling

NSGA-II

Q-Learning algorithm

ABSTRACT

With the widespread adoption of intelligent transportation equipment such as AGVs in the manufacturing field, the flexible job shop scheduling considering limited transportation resources has increasingly attracted attention. However, current research does not consider various dynamic disturbances in real production scenarios, resulting in lower executability of scheduling solutions. To solve this problem, a dynamic flexible job shop scheduling model with limited transportation resources is established, aiming to minimize makespan and total energy consumption. Considering three types of disturbances: job cancellation, machine breakdown, and AGV breakdown, the corresponding event-driven rescheduling strategy is proposed, and a rescheduling instability index is designed to measure the performance of the rescheduling strategy. A Q-Learning-based NSGA-II algorithm (QNSGA-II) is proposed. By learning the feedback historical search experience, it adaptively selects the appropriate neighborhood structures for local search; and a hybrid initialization strategy tailored to the problem characteristics is designed to improve the optimization performance of the algorithm. Through simulation experiments, the effectiveness of the rescheduling strategies and the superiority of the QNSGA-II algorithm in solving such problems are validated.

1. Introduction

Multi-variety and small-batch production is one of the most important production modes in today's manufacturing industry. In response to its complex production planning and scheduling characteristics, many companies have established flexible manufacturing systems (FMS) that integrate information and equipment [1]. As the basis of the flexible job shop production system, how to reasonably arrange its scheduling solution in FMS affects the process of manufacturing to a great extent, so the flexible job shop scheduling problem (FJSP) has been paid more and more attention [2]. Meanwhile, Automated Guided Vehicles (AGVs) are widely employed as crucial intelligent logistics equipment in FMS [3]. There is a strong coupling relationship between the AGVs, the machines and the processing sequence of the job. The former will affect the start time of subsequent operations of the job, and the latter will also affect the selection of AGV [4]. In practical manufacturing systems, the lack of rational and efficient transportation solutions not only leads to increased transportation and time costs but also escalates AGV energy consumption and charging frequency. Thus, considering the flexible job shop scheduling problem with transportation resources (FJSP-T) has become a research hotspot in recent years [5].

Most of the existing research on FJSP-T does not take into account the impact of disturbances [6]. However, in the current complex and dynamic manufacturing scenarios, traditional job shop scheduling methods are difficult to meet the manufacturing demands under various interference events. Continuing to execute the initial solution under disturbance events will seriously reduce the stability of the overall scheduling and the efficiency of the subsequent manufacturing process [7,8]. Especially for more complex scheduling problems such as FJSP-T, when disturbance events occur, it is necessary to replan the machine and AGV at the same time to ensure that the whole scheduling system can respond to various disturbance events in time [9,10]. Therefore, this paper constructs a dynamic flexible job shop scheduling problem with limited transportation resources (DFJSP-T). Based on the characteristics of DFJSP-T, we investigate the disturbance events of the three parts of job, machine and AGV respectively, and propose an event-driven rescheduling strategy for each of the three types of disturbance events.

DFJSP-T is an extension of the classical FJSP, which has been proved to be an NP-hard problem [11]. In addition, most FJSPs involve the simultaneous optimization of multiple objectives [12]. In recent years, the multi-objective optimization problems (MOPs) have

* Corresponding author.

E-mail address: wubin@njtech.edu.cn (B. Wu).

<https://doi.org/10.1016/j.swevo.2024.101658>

Received 7 May 2024; Received in revised form 23 June 2024; Accepted 6 July 2024

Available online 18 July 2024

2210-6502/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

garnered significant scholarly interest, and many scholars have constructed meta-heuristic algorithms to solve MOPs [13]. Among them, the non-dominated sorting genetic algorithm (NSGA-II) characterized by natural evolution has been considered as one of the optimal algorithms for solving MOPs [14]. However, most meta-heuristic algorithms in the existing literature only perform local search strategies through the polling mode, which makes the search have a certain blindness [15]. In contrast, *Q*-Learning, as a reinforcement learning method, maximizes the target reward by using the learning strategy for the agent to select the appropriate action in the interaction with the environment. Related literature shows that the meta-heuristic algorithm combined with *Q*-Learning can obtain better solutions than the traditional meta-heuristic algorithm [16].

In summary, this paper establishes a dynamic scheduling model for flexible job shop considering transportation resources and proposes a *Q*-Learning based NSGA-II (QNSGA-II) algorithm. Specific contributions are as follows:

(1) A mixed-integer dynamic scheduling model for the flexible job shop with limited transportation resources is established. The makespan and total energy consumption (TEC) are taken as the optimization objectives, and the charging constraints of AGV are considered. Three rescheduling strategies are proposed to solve the problems of order cancellation, machine breakdown and AGV breakdown, and a rescheduling instability index is designed to measure the performance of the rescheduling strategies.

(2) A NSGA-II optimization algorithm based on *Q*-Learning is proposed. A hybrid population strategy is designed to construct an initial population by combining three problem-oriented subpopulations to improve the convergence speed and diversity of the algorithm. *Q*-Learning is employed to guide the selection of neighborhood structures in local search, and six neighborhood search operators are constructed as executable actions.

The rest of the paper is as follows. Section 2 reviews the literature on FJSP-T, dynamic scheduling, *Q*-Learning and meta-heuristic algorithms. Section 3 designs the scheduling model of FJSP-T. Section 4 proposes rescheduling strategies to deal with three disturbance events and a rescheduling instability index. Section 5 describes the improved QNSGA-II in detail. Section 6 demonstrates the simulation experiments and analysis results of the dynamic scheduling model and QNSGA-II. The summary of this article is provided in Section 7.

2. Literature review

This section provides a concise review of previous research in three domains: FJSP-T, dynamic scheduling, *Q*-Learning and meta-heuristics.

2.1. FJSP-T

As the production scale increases, the transportation tasks become more intensive, and it is unrealistic to simply consider the FJSP with infinite transportation resources. In order to simulate the actual production scenarios, some researchers have considered limited transportation resources constraints in FJSP.

FJSP-T was first proposed by Deroussi and Norre [17], who designed an iterated local search algorithm and built a benchmark for FJSP-T. Pan et al. [18] proposed a learning-based multi-population evolutionary optimization (LMEO) to deal with FJSP-T. Different subpopulations are designed and an improved selection method is proposed to choose appropriate individuals during the iteration of the algorithm. Liu et al. [19] discussed the AGV and machine integrated scheduling problem under conflict-free paths, proposed a self-learning genetic algorithm, and two different vehicle dispatching strategies are designed. Homayouni and Fontes [6] proposed a scheduling model and designed a late acceptance hill-climbing algorithm. Yan et al. [20] investigated FJSP-T under the condition of limited transportation of

digital twin workshop, a novel three-segment encoding scheme is proposed to improve the performance of genetic algorithm in solving the minimization of makespan problem. Liu et al. [10] proposed a new job insertion FJSP-T and designed a two-stage multi-objective MILP model, and an adaptive large neighborhood search (MOALNS) algorithm is proposed, the validity of MOALNS is verified by the experiments. Xu et al. [21] designed an effective heuristic algorithm (EHA) to solve the multi-objective MILP model in order to minimize TEC and makespan as the objectives of green FJSP-T. Similarly, Li et al. [22] proposed an improved Jaya (IJaya) algorithm to solve FJSP-T with the goal of simultaneously optimizing the TEC and makespan, and developed several problem-oriented local search operators to perform exploitation tasks. LYU et al. [23] considered the AGV path conflict and the optimal number of AGVs in FJSP-T, and proposed an improved genetic algorithm combining time window and Dijkstra algorithm. Kumar et al. [24] designed a novel evolution algorithm to solve the FJSP-T for minimizing the makespan, and proposed a vehicle allocation algorithm and machine selection heuristic embedded into the DE method. Nour et al. [25] proposed a hybrid meta-heuristic based on the cluster holographic multi-agent model to minimize the makespan of FJSP-T.

2.2. Dynamic scheduling

According to different strategies to deal with disturbance events, dynamic scheduling can be roughly classified into three categories: completely-reactive scheduling(CRS), robust proactive scheduling(RPS) and predictive-reactive scheduling(PRS).

Completely-reactive scheduling is a strategy for manufacturing systems to generate scheduling solutions immediately according to real-time states and information. Luo [26] used DRL to solve the DFJSP of new job insertion in the continuous job shop state, and proposed six composite scheduling rules. Cai et al. [27] proposed a real-time information update mechanism and constructed a dynamic job shop scheduling model considering logistics factors. Ghaleb et al. [28] analyzed the effect of using real-time shop floor updates on scheduling decisions in flexible job shop production processes and developed three specific heuristics to deal with order arrivals and machine breakdown in FJSP. In addition, Luo et al. [29] applied RFID technology to obtain real-time data of the production process, and on this basis developed a multi-period hierarchical scheduling mechanism to coordinate and control the production schedule and workload. Li et al. [30] studied FJSP-T based on real-time data, and used the hybrid deep Q network to deal with the dynamic disturbance of machine breakdown and new job insertion.

Robust proactive scheduling means to develop an effective scheduling solution in advance according to the frequent disturbance factors to prevent frequent rescheduling. Liu et al. [31] considered robustness and stability in single-machine scheduling with random machine breakdown. The right-shift rearrangement strategy is adopted to avoid sequence deviation, but this method has shortcomings in resource utilization. Duan et al. [8] proposed a rescheduling strategy considering the overall system effectiveness to study the dynamic scheduling problem including machine breakdown and new job arrivals, and new evaluation indexes were designed from the perspectives of system and processing tasks. Liu et al. [10] discussed FJSP with job insertion under traffic resource constraints, proposed a two-stage multi-objective MILP model, and introduced instability minimization to deal with the impact of disturbances. Yang et al. [32] discussed the dual-objective FJSP with machine breakdown, and developed an alternative measurement index through extreme learning machine to measure the impact of perturbations on system robustness.

Predictive reactive scheduling generates an initial scheduling solution at the beginning, and modifies the initial scheduling solution when disturbance events occur to reduce disturbance to the scheduling objective. The rescheduling methods mainly include left/right shift rescheduling, partial rescheduling and complete rescheduling. Zhu

Table 1

Literature review of DFJSP.

Strategy	AGV	Disturbances	Constraints	Objectives	Solving method	Refs.
CRS	×	Job insertion	×	Total tardiness	Deep Q network	[26]
	✓	Machine breakdown, job insertion	×	Customer satisfaction, equipment utilization, TEC	Simulation-based model with new type	[27]
	×	Machine breakdown, job arrival	×	Total weighted tardiness	Hybrid genetic algorithm	[28]
	×	Job arrival	×	Total earliness, total tardiness, makespan	Multi-period hierarchical scheduling mechanism	[29]
RPS	✓	Machine breakdown, job insertion	×	Makespan, TEC	Hybrid deep Q network	[30]
	×	Machine breakdown, job arrival	×	TEC, makespan, comprehensive reusability	Particle swarm arithmetic optimization	[8]
	✓	Job insertion	×	Makespan, workload balance, stability	Adaptive large neighborhood search algorithm	[10]
	×	Machine breakdown	×	Total weighted tardiness	Two-stage genetic algorithm	[31]
	×	Machine breakdown	×	Makespan, robustness	NSGA- II combined with RMc	[32]
	×	Job arrival	×	Discontinuity rate, makespan deviation, sequence deviation	Improved particle swarm optimization algorithm	[7]
PRS	×	Job cancellation	×	Makespan, TEC	Reformative memetic algorithm	[33]
	×	Machine breakdown, job arrival	Fuzzy processing time, variable machine speed	Makespan, TEC, average agreement index	Q-Learning based immune algorithm	[34]
	×	Machine breakdown	×	Makespan, expected mean tardiness	Genetic algorithm	[35]
	×	Machine breakdown, job insertion	×	Makespan, tardiness	VNS and artificial neural network	[36]
	✓	Job cancellation, machine breakdown, job insertion	×	Makespan	Hybrid artificial bee colony algorithm	[37]
	✓	Job cancellation, machine breakdown, AGV breakdown	AGV charging constraint	Makespan, TEC, rescheduling instability	Q-Learning based NSGA-II	our work

et al. [33] proposed a reformative memetic algorithm to solve the problem of job cancellation in distributed FJSP, and discussed the solution of job cancellation under different processing degrees of jobs. Chen et al. [34] adopted a predictive reactive rescheduling method for fuzzy scheduling problems under dynamic disturbances, and proposed an offline-based MILP model and an online-based rescheduling model. Gholami et al. [35] combined simulation and genetic algorithms to solve the dynamic scheduling problem of machine breakdown and generated a new scheduling solution through event-driven strategies and right-shift heuristics. Likewise, Adibi et al. [36] considered two event-driven rescheduling problems, random job arrivals and machine breakdown, and used artificial neural networks to adjust VNS parameters to improve algorithm performance. Li et al. [37] developed a hybrid artificial bee colony algorithm and used corresponding strategies for three kinds of emergency events.

2.3. Q-Learning and meta-heuristics

At present, some scholars have begun to notice that combining *Q*-Learning and meta-heuristic algorithms to solve combinatorial optimization problems can improve the performance of algorithms. In the algorithms, *Q*-Learning is mainly applied to adaptively adjust key parameters and strategy selection. Chen et al. [38] used *Q*-Learning to adaptively adjust the crossover and mutation operations in their GA. Shahrabi et al. [39] used *Q*-Learning to select three parameters for variable neighborhood search (VNS): the number of external iterations, the number of internal iterations, and the acceptance threshold. In addition, Yao et al. [40] proposed a MA combined with deep Q-network to adjust the crossover based on the population evolution. On the other hand, Zhang et al. [41] used the MA algorithm based on deep *Q*-Learning to solve the integrated scheduling problem, and designed local search operators based on critical blocks. Zhao et al. [42] used the *Q*-Learning method to select an appropriate low-level heuristic strategy according to historical information. Li et al. [43] improved the artificial bee colony (ABC) algorithm with *Q*-Learning, designed a set

of neighborhood structures based on knowledge and problem characteristics, and used *Q*-Learning to adaptively select the neighborhood search operator in each iteration. Similarly, Wang et al. [44] proposed a *Q*-learning based ABC algorithm, which uses 12 types of population quality as states and 6 neighborhood search operators as actions. Wang et al. [45] established a dynamic rescheduling model based on multi-agent technology, and developed a clustering and weighted *Q*-Learning algorithm to select the optimal strategy in the dynamic scheduling process. Cheng et al. [46] proposed a bi-criteria *Q*-Learning algorithm based on Pareto and index selection (QHH-BS). In each iteration, the algorithm can adaptively select an optimizer from three operators.

Table 1 provides a summary of the literature concerning DFJSP and outlines the contributions of this paper. Based on the literature above, we can draw the following conclusions:

(1) The dynamic scheduling problem has received widespread attention, but the dynamic scheduling problem in FJSP-T and the impact of AGV charging constraints are rarely considered.

(2) Machine breakdown and job changes (such as job cancellation and insertion) are the most frequent disturbance events in DFJSP. In addition, there has been no relevant research on AGV breakdown thus far.

(3) There are some literatures that combine the *Q*-Learning algorithm and the metaheuristic algorithm, but there is no research on improving NSGA-II based on *Q*-Learning to solve dynamic FJSP-T.

Based on the above analysis, a rescheduling model for FJSP-T considering AGV charging constraints is constructed. We investigate two frequent disturbance events: job cancellation and machine breakdown, as well as the newly introduced disturbance event of AGV breakdown in FJSP-T, and propose three event-driven rescheduling strategies. An improved NSGA-II combined with *Q*-Learning is innovatively developed to solve DFJSP-T.

3. FJSP-T model

FJSP-T can be described in detail as follows: a job $i \in \{1, 2, \dots, N\}$ is processed on machines $m \in \{1, 2, \dots, M\}$, each job i has O_i operations,

Table 2

Notations.

Symbol	Description
i, d, k	Index of jobs
j, h, f	Index of operations
$t_{i,j,m}$	Processing time of operation O_{ij} on machine m
C_i	Completion time of job i
C_m	Completion time of machine m
C_{max}	Makespan
P_m^{busy}	Unit processing energy consumption of machine m
P_m^{idle}	Unit idle energy consumption of machine m
TEC	Total machine energy consumption
E_{idle}	Total idle energy consumption of all machines
E_{busy}	Total processing energy consumption of all machines
$T_{i,j,m}^s$	Processing start time of O_{ij} on machine m
$T_{i,j,a}^{l,e}$	Load end time of AGV a transport O_{ij}
$T_{i,j,a}^e$	End processing time of O_{kf} on machine m
$T_{i,j,a}^{l,s}$	Load start time of AGV a transport O_{ij}
$T_{i,j,a}^{u,e}$	No-load end time of the AGV a transport O_{ij}
$T_{i,j-1,m}^e$	Completion time of $O_{i,j-1}$ on machine m
$T_{i,j,a}^{u,s}$	No-load start time of the AGV a transport operation O_{dh}
$t_a^{m \rightarrow c}$	Transportation time of AGV a from machine m to charging area
t_a^{charge}	Charging time of AGV a
T_a^{charge}	Charging start time of AGV a
TQ_a	Maximum power threshold of AGV a
LQ_a	Minimum power threshold of AGV a
$Q_{i,j,a}^{l,e}$	End-of-load power of O_{ij} transported by AGV a
$Q_{i,j,a}^{u,s}$	No-load starting power of O_{dh} transported by AGV a
$P_a^{m \rightarrow c}$	Unit no-load transportation energy consumption of AGV a
$P_a^{l,e}$	Unit load transportation energy consumption of AGV a
T_a^{charge}	The charging rate of AGV a
$x_{i,j,m}$	1 if O_{ij} is processed on machine m , otherwise 0
$y_{i,j,a}$	1 if O_{ij} is transported by AGV a , otherwise 0
$w_{i,j,a}$	1 if AGV a needs to be charged after transporting O_{ij} , otherwise 0

and each job is transported through AGV $a \in \{1, 2, \dots, A\}$. O_{ij} denotes the j_{th} operation of job i . The operation and AGV start from the loading and unloading area. According to the process route of each operation, the transportation task of each operation O_{ij} can be transported by any AGV, and each operation O_{ij} can be processed by any one of its optional machines. After the operation processing is completed, the AGV transports the operation to the unloading area. When the AGV is below the minimum power threshold, it needs to go to the charging area for charging. The schematic diagram of FJSP-T is shown in Fig. 1. More precisely, the problem consists of three sub-problems:

- (1) the scheduling sequence problem (OS).
- (2) Assign operations to appropriate machines (MS).
- (3) Use the appropriate AGV transport operation (AS).

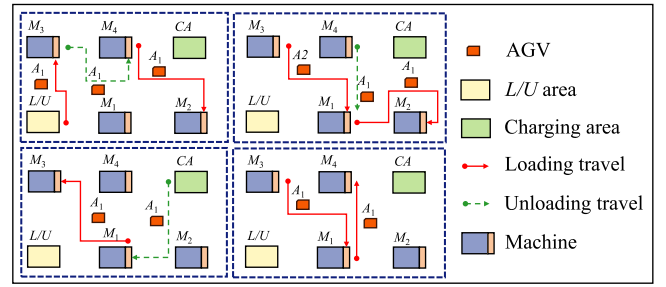
For the FJSP-T model proposed in this paper, the following assumptions are made:

- (1) A job can only be transported by one AGV at the same time.
- (2) Each operation of the job can only be processed on one machine at the same time;
- (3) the energy consumption per unit time of AGV is different in no-load and load states;
- (4) The minimum power threshold of AGV can complete a furthest distance task with the highest energy consumption and return to the charging area. There is no case of power loss midway, and AGV is not allowed to return to the charging area when it is loaded;
- (5) Assume that the AGV charging process is not interrupted until the power reaches the highest threshold;
- (6) Path conflict is not considered.

The notations used in this paper are given in Table 2. The FJSP-T model with the optimization objective of minimizing the makespan and TEC is established as follows.

$$\min C_{max} = \max_{1 \leq i \leq N} C_i, \forall i. \quad (1)$$

$$\min TEC = E_{busy} + E_{idle}. \quad (2)$$

**Fig. 1.** An illustrative example of FJSP-T.

$$E_{busy} = \sum_{m=1}^M \sum_{i=1}^N \sum_{j=1}^{O_i} t_{i,j,m} \cdot x_{i,j,m} \cdot P_m^{busy}. \quad (3)$$

$$E_{idle} = \sum_{m=1}^M (C_m - \sum_{i=1}^N \sum_{j=1}^{O_i} t_{i,j,m} \cdot x_{i,j,m}) \cdot P_m^{idle}. \quad (4)$$

$$\sum_{m=1}^M x_{i,j,m} \leq 1, \forall i, j. \quad (5)$$

$$\sum_{a=1}^A y_{i,j,a} \leq 1, \forall i, j. \quad (6)$$

$$T_{i,j,m}^s \geq \max\{T_{i,j,a}^{l,e}, T_{k,f,m}^e\}, \forall i, j, k, f, m, a. \quad (7)$$

$$T_{i,j,a}^{l,s} \geq \max\{T_{i,j,a}^{u,e}, T_{i,j-1,m}^e\}, \forall i, j, m, a. \quad (8)$$

$$T_{i,j,a}^{u,s} \geq T_{i,j,a}^{l,e} + w_{i,j,a} (t_a^{m \rightarrow c} + t_a^{charge}), \forall i, j, d, h, m, a. \quad (9)$$

$$t_a^{charge} = (TQ_a - (Q_{i,j,a}^{l,e} - t_a^{m \rightarrow c} \cdot P_a^u)) / P_a^{charge}, \quad \forall i, j, k, r. \quad (10)$$

$$Q_{k,f,a}^{u,s} = (1 - w_{i,j,a}) \cdot Q_{i,j,a}^{l,e} + w_{i,j,a} \cdot TQ_a, \forall i, j, a. \quad (11)$$

$$T_a^{charge} = T_{i,j,a}^{l,e} + t_a^{m \rightarrow c}, \forall i, j, k, a. \quad (12)$$

Eqs. (1), (2) are the two objective functions of the model, which are makespan and TEC, respectively. Eqs. (3), (4) describe in detail the energy consumption when the machine is processing and idle. Eqs. (5), (6) respectively indicate that a job can only be transported by one AGV and processed by one machine at the same time. Eq. (7) represents the job processing time constraint. The start processing time of the j_{th} operation of job i depends on the arrival time of the job as well as the completion time of the previous job on machine m . Eq. (8) represents the load start time constraint of AGV a . The start time of AGV a performing the transport task of operation O_{ij} depends on the end time of the no-load of AGV a as well as the end time of the previous operation of job i . Eq. (9) represents the no-load start time of AGV transportation operation. After an AGV completes the previous delivery task, if there is no need to charge, the AGV waits for the next task in the original position. If charging is required, the AGV needs to go to the charging area to charge to the highest power threshold before starting the next task. Eq. (10) represents the charging time. Eq. (11) indicates that if the AGV goes to the charging area for charging after the previous delivery task, the power of the AGV will be updated to the highest power. Otherwise, the current power is the remaining power after the completion of the previous transport task. Eq. (12) respectively represents the charging start time when the AGV arrives at the charging area.

4. Rescheduling strategy and index

4.1. Rescheduling strategy

In dynamic scheduling problems, when disturbance events occur, appropriate rescheduling heuristics need to be employed to address

them. Due to the inclusion of AGV scheduling with charging constraints in DFJSP-T, traditional rescheduling strategies may not be suitable for the dynamic scheduling of production systems. Therefore, this paper designs three rescheduling strategies for the three types of disturbance events in DFJSP-T, including job cancellation, machine breakdown, and AGV breakdown, and compares them with the complete rescheduling strategy in 6.5. In order to determine the status of jobs, machines and AGVs after different disturbance events, Algorithm 1 is designed to obtain rescheduling information.

Algorithm 1: Obtain Rescheduling Information

Input: Machine and AGV schedule $\tilde{M}_m, \tilde{A}_a$, event occur time T_d , breakdown equipment M_b, A_b , machine and AGV maintenance time mt_m, at_m

Output: Earliest available time of machine and AGV ET_m, ET_a , remaining operations O_{ij}^n , job and AGV position $i_{position}, a_{position}$

```

for each  $m$  in  $\tilde{M}_m$  do
    if  $T_{i,j,m}^s < T_d < T_{i,j,m}^e$  then
        Machine  $m$  stops after the current operation;
    end
    else if  $T_d = T_{i,j,m}^e$  or machine is idle at time  $T_d$  then
        Machine  $m$  stops immediately;
    end
     $ET_{m \neq M_b} \leftarrow T_{stop}^m$ ,  $ET_{m=M_b} \leftarrow T_{stop}^m + mt_m$ ;
end
for each  $a$  in  $\tilde{A}_a$  do
    //AGV is in a transport task or a charging task;
    if Task start time  $< T_d <$  Task end time then
        AGV  $a$  stops after the current task;
    end
    else if  $T_d =$  Task end time or AGV is in idle then
        AGV  $a$  stops immediately;
    end
     $ET_{a \neq A_b} \leftarrow T_{stop}^a$ ,  $ET_{a=A_b} \leftarrow T_{stop}^a + at_m$ ;
end

```

Determine $i_{position}$ and $a_{position}$;
Record the number of unfinished operations O_{ij}^n ;

To clearly illustrate the DFJSP-T, we use an example, shown in Fig. 2(a), which is the gantt chart of an FJSP-T containing three jobs, three machines, and two AGVs. The operands of J_1, J_2 , and J_3 are 4, 4, and 4, respectively. The makespan is 34, the TEC is 276.8, and the completion time of J_1, J_2 , and J_3 is 34, 29, and 28, respectively. C indicates that the AGV is in charging state.

RS_1: It states that in the case of job cancellation, the disturbance degree to the scheduling system is small, and the initial scheduling solution can be kept unchanged in the whole. It can be found that when the old job is canceled, the original AGV and machine resources will be released, and the rescheduling solution will usually change to a smaller time and energy consumption. RS_1 tries to keep the jobs processed on each machine and the processing sequence unchanged and does not limit the change of AGV. Therefore, the AGV used in the rescheduling solution may or may not change, depending on whether changing the AGV transportation can reduce the target value.

Algorithm 2: Rescheduling Strategy 1

Input: Rescheduling information, initial solution, canceled job J_c

Output: Rescheduling solution

Delete all operations of job J_c in O_{ij}^n ;

Delete the machine used for all operations of job J_c ;

OS', MS' maintains the order in initial solution;

$AS' \leftarrow$ random select;

The rescheduling solution is obtained using QNSGA-II;

As shown in Fig. 2(b), job 2 is cancelled at time 15, which indicates that the subsequent O_{23}, O_{24} will not be processed anymore. The

second operation of job 2 continues processing on machine M_2 until completion. The order of operations on all machines remains the same as in the original solution.

RS_2: When the disturbance event is machine breakdown, it will disturb the initial solution more than the job cancellation, so the rescheduling strategy as follows is considered.

For the operations on the non-broken machine, the original machine is still used for processing, so as to reduce other scheduling costs caused by the transformation of the processing machine. The job on the broken machine will continue processing on the machine after the machine repair is completed; In addition, the processing machine is randomly arranged for the subsequent unprocessed operation process of the broken machine. Note that changes in the AGV used for transport after rescheduling have little impact on the system, so the AGV used for all jobs in the rescheduling process can be changed to ensure that jobs are transported by the most suitable AGV and enter the process as soon as possible.

Algorithm 3: Rescheduling Strategy 2

Input: Rescheduling information, initial solution, M_b

Output: Rescheduling solution

if the operation O_{ij} is processed on M_b then

Randomly inserted into a machine on the optional machine set;

else

O_{ij} continues to process on the initial machine, and all operations maintain the initial order;

end

$AS' \leftarrow$ random select;

The rescheduling solution is obtained using QNSGA-II;

As shown in Fig. 2(c), when machine M_1 breaks down at time 10 and the maintenance time is 5, for the jobs on M_2, M_3 , the original machine is still kept for processing. When the breakdown occurs, O_{21} is still processing on M_1 , so the second half of the processing of O_{21} will be carried out immediately after the maintenance of machine M_1 . Due to the delay of the completion time of the second lane operation of job 2 due to the breakdown of M_1 , the scheduling solution of A_2 is modified to transport job 3 in advance and then transport job 2.

RS_3: For the DFJSP-T problem in this paper, AGV breakdown will also have an impact on the overall scheduling solution. Usually AGV breakdown may occur in two situations. First, the AGV is in transport operation, and the second AGV is in the process of no-load movement. In the case of AGV load transportation, the jobs currently transported by the AGV will continue to be transported by the AGV after the repair of the AGV, so the arrival time of the job will be affected. Based on this consideration, we decide that all jobs will still be processed on the original machine, but the processing sequence of the jobs can be exchanged. The AGV used by all jobs can also be adjusted to maximize the utilization of transportation resources in the event of an AGV breakdown. For the case of no-load transport movement of AGV, when the maintenance of AGV is completed, it will continue to run the end point of this sub-movement task in the initial solution, and then wait for the allocation of new tasks according to the new solution.

Algorithm 4: Rescheduling Strategy 3

Input: Rescheduling information, initial solution, A_b

Output: Rescheduling solution

The processing machine MS' is constant for all operations, the processing order OS' can be changed;

$AS' \leftarrow$ random select;

The rescheduling solution is obtained using QNSGA-II;

As shown in Fig. 2(d), AGV 1 fails next to machine M_3 at time 1, and the power of AGV 1 reaches the lowest threshold. After a maintenance time of 5, the AGV continues to the charging area for charging. All jobs are still fixed to the original machine for processing, and it can be found that in the rescheduling solution, all operations still maintain the original processing order.

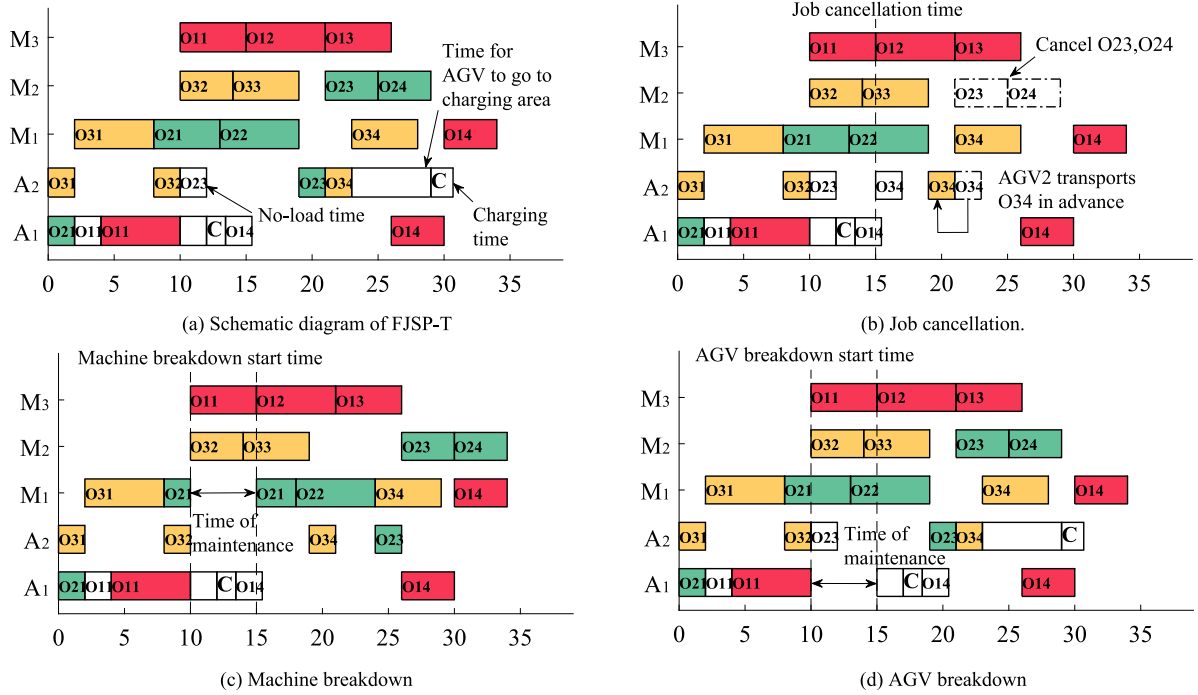


Fig. 2. Gantt chart of the example of DFJSP-T.

4.2. Rescheduling index

In DFJSP-T, a very important point to evaluate the quality of rescheduling solution is the instability of rescheduling. Rescheduling instability is generally defined as the change of the rescheduling solution with respect to the initial solution in all aspects. To measure the instability of a rescheduling solution, this paper defines a new index, rescheduling instability (RSI), which consists of the following three sub-indicators.

(1) Machine change rate (MCR)

$$\min MCR = \min \frac{\tilde{n}_{op}}{\min\{n_{ori}, n_{now}\}} \quad (13)$$

In Eq. (13), if the disturbance event is machine breakdown and AGV breakdown, MCR is equal to the ratio of the number of processes changed by the processing machine on the initial scheduling solution to the number of processes in the initial scheduling solution. If the disturbance event is a job cancellation, MCR measures the ratio of the number of processes in the rescheduling solution that changed the machine relative to the initial solution to the number of processes in the rescheduling solution.

(2) AGV energy consumption balance rate (σ_a)

$$\min \sigma_a = \min \sqrt{\frac{\sum_{a=1}^A (E_a - \bar{E}_a)^2}{A}} \quad (14)$$

In this equation,

$$\bar{E}_a = \frac{\sum_{a=1}^A E_a}{A} \quad (15)$$

σ_a calculates the standard deviation of energy consumption of all AGVs in the system to measure the degree of difference in energy consumption of each AGV during transportation. The smaller σ_a is, the more balanced job distribution of AGV is indicated. Here, E_a represents the total energy consumption of AGV a .

(3) Average processing start-time deviation (APSD)

$$\min APSD = \min \frac{\sum_{i=1}^n \sum_{j=1}^{O_i} |\hat{x}_{ij} - x_{ij}|}{n} \quad (16)$$

APSD represents the average change in the job start processing time in the rescheduling solution relative to the initial scheduling solution, $\hat{x}_{ij}$ represents the start processing time of operation O_{ij} in the rescheduling solution, x_{ij} represents the start processing time of operation O_{ij} in the initial scheduling solution, n represents the operation after the disturbance event quantity.

$$\min RSI = \lambda_1 \cdot MCR + \lambda_2 \cdot \sigma_a + \lambda_3 \cdot APSD \quad (17)$$

Considering the disturbance degree of the three subsystems of job, machine and AGV in DFJSP-T after dynamic events, the weight coefficients are given after normalization respectively, and finally the index to measure the instability of the rescheduling solution (RSI) is formed. It can be seen from the Eq. (17) that a higher RSI value indicates poorer robustness of the rescheduling solution.

5. Q-Learning based NSGA-II

5.1. Encoding and decoding

The proposed DFJSP-T involves scheduling three resources: jobs, machines, and AGVs. A three-segment coding method is utilized to encode the problem. As depicted in Fig. 3, the first segment is the job coding segment (OS), where each element represents the job number of its corresponding operation. The j_{th} occurrence of a job number signifies the j_{th} operation of that particular job. The second segment is the machine coding segment (MS), the number on MS represents the m_{th} machine in the machine candidate set of the corresponding job. MS is arranged based on both the job number and operation number in ascending order. Lastly, we have AGV coding segment (AS) which follows a similar arrangement as MS, indicating that each process of a job is transported by its respective AGV. This encoding approach ensures simplicity and flexibility while guaranteeing feasibility of generated chromosome solutions during subsequent operations, thereby eliminating complex chromosome repair procedures.

A left shift insertion active decoding rule is designed for DFJSP-T. The decoding process mainly includes three steps: (1) power check, (2) charging process, and (3) left shift insertion. The decoding process is described in detail in Algorithm 5.

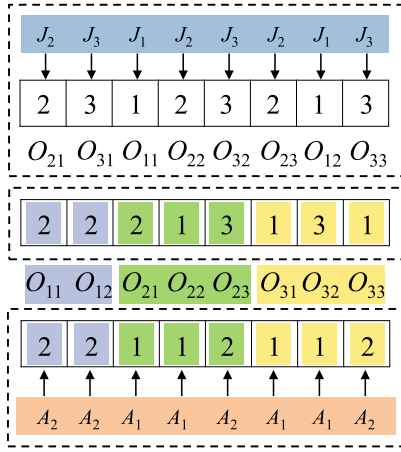


Fig. 3. Encoding of a three-segment code.

5.2. Hybrid initialization strategy

The quality of the initial population directly affects the convergence speed and performance of the algorithm, and individuals are usually generated by random initialization because there is no prior information. However, sometimes individuals are not evenly distributed in the search domain, which may make individuals far away from the global optimal solution and lead to low convergence speed. In the process of population initialization, chaotic variable search has more advantages than random search [47]. The equation of tent chaotic mapping (TCM) is as follows:

$$x_{t+1} = \begin{cases} 2x_t, & 0 \leq x_t \leq 0.5 \\ 2(1 - x_t), & 0.5 < x_t \leq 1 \end{cases} \quad (18)$$

Algorithm 5: Decoding Strategy

Input: OS , MS , AS , initial parameters

Output: Solution

```

for each  $O_{ij}$  in  $OS$  do
    Find  $M_k$  and  $agv_a$  in  $MS$  and  $AS$ ;
    for each  $AGV$  do
        if  $Q_a \leq LQ_a$  then
             $AGV$  immediately goes to charging;
            Update the data of  $agv_a$ ;
        end
    end
    Get  $T_{k,f,m}^e$  and  $T_{i,j,a}^{l,e}$  and calculate  $T_{i,j,a}^{l,e}$  by Eq. (7);
    Calculate time required for transportation;
    Update the data of  $agv_a$ ;
    Get  $T_{i,j,a}^{u,e}$  and  $T_{i,j-1,m}^e$  to calculate  $T_{i,j,a}^{l,s}$  by Eq. (8);
    Calculate  $O_{ij}$  processing duration time;
    Inserted into the earliest available period of  $M_k$ ;
    Update the data of  $M_k$ ;

```

end

To ensure the efficacy of TCM for small periodic and unstable periodic points generated in the iterative sequence, random variable $\text{rand}(0,1)$ is introduced. Based on the characteristics of DFJSP-T and tent chaotic map, we design a variety of heuristic rules for each coding segment, and constructs a hybrid initialization strategy (HIS) containing three sub-populations.

The size of the first sub-population is 60% of the population size, and the three coding segments of OS , MS and AS of the first sub-population are initialized using tent chaotic map to form a sub-population that is evenly distributed in space.

The second sub-population is 20% of the population size. For the individuals in this population, tent chaotic map is used to generate the OS coding segment, and each operation is processed using the machine with the shortest current cumulative processing time and transported by the AGV closest to the job. The purpose of this sub-population is to improve machine utilization and reduce the energy consumption of $AGVs$ when unloaded.

The remaining 20% individuals are used as the third sub-population, and the operation on each individual in the population prefers the machine with the least processing energy consumption with a certain probability. In the OS coding segment, the AGV with the current minimum load is selected for transportation, and the load of the AGV is measured by the electricity consumption of the AGV in this paper. This sub-population ensures a lower TEC as well as load balancing of $AGVs$.

Through the hybrid population strategy, the convergence speed of the iterative process can be increased while ensuring the diversity and distribution of the population. In the subsequent process of solving DFJSP-T, only part of the coding segments need to be initialized again according to the requirements of different rescheduling strategies in Section 4.1. The hybrid initialization strategy is described in Algorithm 6.

Algorithm 6: Hybrid Initialization Strategy

Input: M_{ij} , $opera_set$, pop

Output: Initial solution

Generate the first population by TCM;

Generate $20\% \times pop$ size OS codes by TCM;

for each individual **in** $20\% \times pop$ **do**

for each O_{ij} **in** OS **do**

 Select M_k with the shortest cumulative processing time in

M_{ij} ;

 Update the cumulative processing time of M_k ;

 Select agv_a closest to the job;

 Update the position and power of agv_a ;

end

end

Generate $20\% \times pop$ size OS codes by TCM;

for each individual **in** $20\% \times pop$ **do**

for each O_{ij} **in** OS **do**

if $\text{rand} < 0.5$ **then**

 Select M_k with the minimum energy consumption in

M_{ij} ;

else

 Randomly select a machine M_k in M_{ij} ;

end

 Select agv_a with the lowest cumulative load;

 Update the position and power of agv_a ;

end

end

5.3. Selection, crossover and mutation operator

The role of the selection operator is to retain the genes of excellent individuals, which have high fitness values. By randomly selecting two individuals from the population for comparison, the binary tournament selection operator continuously selects good parent individuals to construct the population of the new generation.

The crossover operation improves the exploration ability of the algorithm. According to the characteristics of DFJSP-T, a crossover mechanism for three-segment coding is proposed. For OS coding segments, the precedence operation crossover [48] (POX) is used to ensure OS legitimacy in the offspring chromosome. The multi-point crossover [49] (MPX) is used for the machine coding segment MS . Because MPX operator is the crossover of the processing machines used in the same process, the generated offspring chromosome MS is a feasible solution.

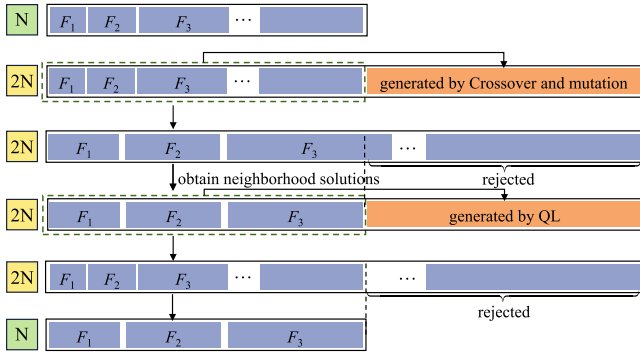


Fig. 4. Schematic diagram of the population structure.

Similarly, the same crossover operator as MS is used in the AGV coding segment AS.

The mutation operator enhances population diversity, enabling the algorithm to escape local optima and explore the solution space more extensively. Similar to the crossover operation, different mutation operators are used according to the characteristics of the three-segment encoding. In OS, the process positions of two different jobs are exchanged arbitrarily. For MS and AS, replace the current device with another device from the set of available devices for the current process.

5.4. Non-dominated sorting and crowding distance sorting

In MOPs, for two solutions x and y , x dominates y if x outperforms y under all objective functions. The process of non-dominated solution ranking is to divide all solutions into different levels according to their dominance relationship. For the solutions in each level, by giving priority to the solutions with low crowding degree, the diversity in the level can be preserved as much as possible to prevent the algorithm from falling into local optimal solutions. Both techniques are used in each iteration, and the population structure in the Q-NSGA-II iteration is shown in Fig. 4.

5.5. Q-Learning based local search

Q-Learning algorithm has shown excellent performance in solving various heuristic problems, it usually consists of five elements, respectively is the agent, environment, action set, state set and reward function, forming a quintuple (A, E, C, S, R) . At each decision time point, the agent selects the appropriate action based on the current state of the environment, and then updates the table of state-action mapping (Q -table) based on the immediate rewards given by the environment. The Q -table records the long-term expected reward for each state-action mapping, thus guiding the agent to make decisions and gradually learn the optimal strategy as it continuously interacts with the environment. The update equation of Q -table is as follows.

$$Q(S_t, a_t) = \alpha [R + \gamma \max_a Q(S_{t+1}, a)] + (1 - \alpha) \cdot Q(S_t, a_t) \quad (19)$$

In Eq. (19), $Q(S_t, a_t)$ represents the Q value generated by taking the action A_t in the current S_t state. α represents the learning rate, γ represents the discount factor, and R is the reward after performing the action A_t . $\max_a Q(S_{t+1}, a)$ represents the highest Q -value for which the next state S_{t+1} takes all actions A .

By combining the learning ability of Q -learning and the global search ability of NSGA-II, the convergence and search efficiency of NSGA-II are improved. Q -learning can be used in local search in solution space to find high quality solutions near pareto frontier faster, thus speeding up the convergence process of the algorithm. The main concepts of Q -learning are defined as follows.

Table 3

State set.

State	Description	State	Description
1	$\gamma_n \in \varphi_1(1)$ $\gamma_n \in \varphi_2(1)$	2	$\gamma_n \in \varphi_1(1)$ $\gamma_n \in \varphi_2(2)$
3	$\gamma_n \in \varphi_1(2)$ $\gamma_n \in \varphi_2(1)$	4	$\gamma_n \in \varphi_1(2)$ $\gamma_n \in \varphi_2(2)$

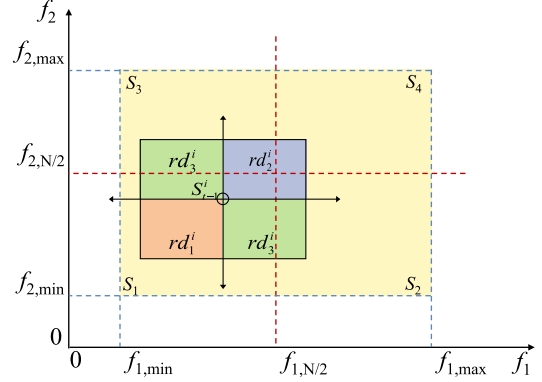


Fig. 5. Diagram of state division and reward setting.

State. The exploitation ability of the algorithm is improved by determining the state in which the current solution is located and selecting the best action for each solution. This paper considers two conflicting objectives, makespan and TEC, so it is necessary to make a corresponding tradeoff between them. As shown in Fig. 5, the fitness values of all N individuals in the population under the f_1 objective are calculated and sorted in ascending order, and the target value located on the median in the f_1 direction is found. The N individuals are divided into two regions with the target value as the limit. Likewise, find the median in the f_2 direction and partition the space. In this way, the whole pareto plane is divided into four regions as states in Q -learning. The detailed definitions of the four states are given in Table 3, where $\varphi_i(1)$ denotes the first part after all individuals are sorted in ascending order according to the i_{th} objective value, and $\varphi_i(2)$ denotes the second part.

Reward. In this paper, we classify reward values into three types based on the dominance relationship between the newly generated solution, denoted as α , and the old solution, represented by β . These categories entail three distinct cases: when the newly generated solution dominates the old one ($\alpha > \beta$), when it is dominated by the old solution ($\alpha < \beta$), and when neither solution dominates the other ($\alpha \simeq \beta$). The detailed reward rd function is defined as follows.

$$rd = \begin{cases} \delta_1 + \sum_{k=1}^K \frac{f_{\max}^k - f_{\min}^k}{f_{\max}^k - f_{\min}^k}, & \text{if } \alpha > \beta \\ 0, & \text{if } \alpha < \beta \\ \delta_2 + \sum_{k=1}^K \frac{f_{\max}^k - f_{\min}^k}{f_{\max}^k - f_{\min}^k}, & \text{if } \alpha \simeq \beta \end{cases} \quad (20)$$

The reward function rd considers the dominance relationship between the newly generated solution and the initial solution, as well as the degree of optimization of the new solution on each objective. This method can assign higher rewards to actions that have better optimization effects on objective values. δ_1 and δ_2 as rewards for measuring the dominance relationship are set to 2 and 1, respectively. In addition, for two solutions that do not dominate each other, if the new solution is better than the old solution in one objective value, but worse than the old solution to a greater extent for another objective value, the reward function will reduce the reward for that action. Such a reward function solves the problem of global exploration capability and local exploitation capability.

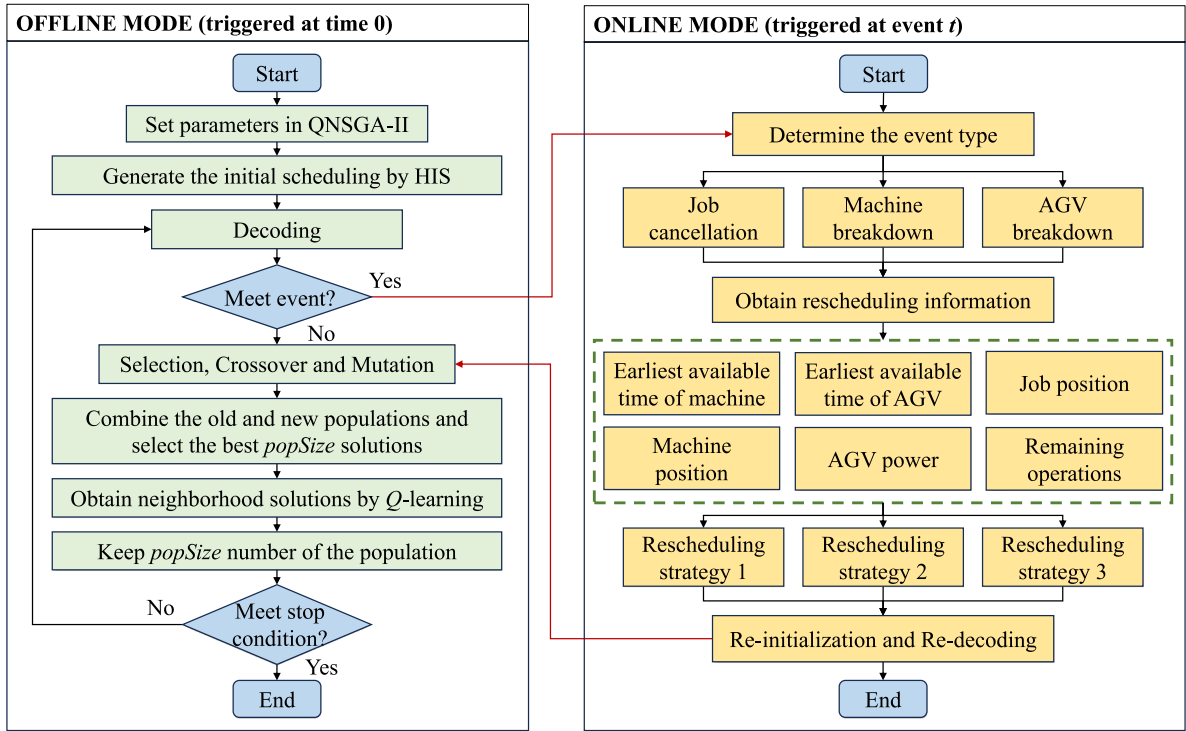


Fig. 6. Framework for solving DFJSP-T model.

Policy. The ϵ -greedy method is used for each individual action. Specifically, when the random probability is less than ϵ , the action corresponding to the largest Q value in the Q -table in the current state is selected. When the random probability is greater than ϵ , then one of the actions is randomly selected. In this paper, a dynamic ϵ function is designed to guide the action selection in different iteration periods. The equation is as follows:

$$\epsilon(g) = \frac{1}{1 + e^{1.1 - \frac{5g}{GEN}}} \quad (21)$$

Eq. (21) shows that in the early iteration process, the algorithm prefers to use more random actions to train the Q -table. As the number of iterations g increases, the agent can select the most appropriate action by using the existing knowledge in the Q -table to enhance the development ability of local search.

Action. In QNSGA-II, actions are defined as six neighborhood structures. The algorithm selects one neighborhood structure to enter the local search in each iteration. The design of the relevant domain structure is as follows.

NS_1: The operation of randomly selecting two adjacent positions and inserting any position in reverse order.

NS_2: Two operations are deleted randomly, and then inserted into the operation coding segment in the order of deletion.

NS_3: Randomly select the machine used in an operation and replace it with any other machine in its optional machine set for processing.

NS_4: Find the machine with the most processing jobs, process any operation from this machine, and modify it to the machine with the least load in the set of optional equipment.

NS_5: On the AGV coding segment, a random point position is used for transportation using other AGVs.

NS_6: The waiting time of a no-load AGV arriving in advance is used for the transportation of other jobs, that is, during the waiting time, a transportation task of other AGVs is replaced by this AGV for transportation, and it is set as the previous task of this no-load task.

5.6. Framework for solving DFJSP-T

The framework for solving DFJSP-T proposed in this paper is shown in Fig. 6. The initial scheduling solution is generated by offline mode. When an disturbance event occurs, the corresponding rescheduling strategy is selected according to the type of disturbance event in online mode, and the new chromosome is re-decoded after obtaining the rescheduling information. It then returns to offline mode and re-generates the subsequent solution using the QNSGA-II algorithm guided by the rescheduling strategy.

6. Experimental analysis

In this paper, we extend the MK benchmark proposed by Brandimarte [50] to construct the EMK benchmark for DFJSP-T. The number of AGVs is determined according to the number of operations, i.e., {job number - AGV number}, and we set three benchmarks of different sizes: {10-3}, {15-4}, {20-5}. All programs are coded and implemented in Matlab 2021b and run on an Intel(R) Core(TM) i9-13900K CPU @ 3.00 GHz with 128-GB RAM.

6.1. Algorithm performance measures

For multi-objective optimization problems, the solutions generated by the algorithm are non-dominated on the pareto front, and therefore cannot be evaluated directly on the basis of the individual objective values. In order to measure the performance of QNSGA-II in solving DFJSP-T, we comprehensively investigate the algorithm in terms of convergence, diversity, and solution quality, using three evaluation criteria, including Hypervolume (HV) [51], Inverted Generational Distance (IGD) [52], and C-metric [53], respectively. HV measures the comprehensive performance of the algorithm, the larger HV, the better the algorithm performance. IGD measures the convergence and diversity of the algorithm. Contrary to HV, smaller IGD of the algorithm is better. C-metric measures the degree of dominance between the solutions produced by two algorithms.

Table 4
Orthogonal table.

Serial number	P_c	P_m	α	γ	Value
1	0.7	0.05	0.1	0.7	0.5352
2	0.7	0.1	0.2	0.8	0.4754
3	0.7	0.15	0.3	0.9	0.4789
4	0.8	0.05	0.2	0.9	0.5513
5	0.8	0.1	0.3	0.7	0.6229
6	0.8	0.15	0.1	0.8	0.6675
7	0.9	0.05	0.3	0.8	0.2994
8	0.9	0.1	0.1	0.9	0.6879
9	0.9	0.15	0.2	0.7	0.4932

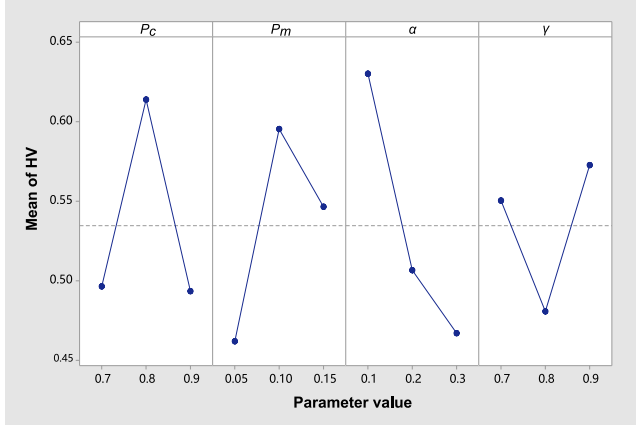


Fig. 7. Main effects plot for HV index.

It is worth noting that the PF_{true} is usually given in advance when computing the IGD, but there is no PF_{true} for DFJSP-T so far. Therefore, we use the non-dominated solutions from the set of solutions obtained by all the comparison algorithms instead of PF_{true} .

6.2. Parameter setting

In order to investigate the effect of parameter settings on the performance of the proposed QNSGA-II, the Taguchi design of experiments methodology is used to analyze four key parameters: crossover probability P_c , mutation probability P_m , learning rate α , and discount factor γ . The orthogonal array L9 (3^4) is shown in Table 4, where the QNSGA-II is repeated 20 times for each parameter combination, and the obtained main effects plot of the mean HV values is shown in Fig. 7.

Fig. 7 shows the horizontal trend of the four parameters with respect to the HV index at the three levels. It can be observed that the QNSGA-II algorithm achieves better performance in terms of HV measure when the parameter levels are set to $P_c = 0.8$, $P_m = 0.1$, $\alpha = 0.1$ and $\gamma = 0.9$.

6.3. Effectiveness of new strategies on QNSGA-II

The main improvement of NSGA-II is the design of the hybrid initialization strategy and the local search based on Q -Learning. In order to test the effectiveness of these improved strategies, we designed three corresponding variants of QNSGA-II. The classical NSGA-II algorithm is denoted as A, which uses the random generation method to construct the initialization population. The NSGA-II based on the hybrid initialization strategy is denoted B. C represents replacing the Q -Learning-based neighborhood search in QNSGA-II with a general neighborhood search, that is, the choice of neighborhood structure is obtained by a random method. The parameter settings of the three algorithms A, B and C are the same as those of QNSGA-II. QNSGA-II and the three variants are run 20 times randomly on each instance. Table 5 shows the obtained HV average and IGD average on each instance for each algorithm.

As shown in Table 5, algorithm A outperforms algorithm B in both the HV average and the IGD average in all instances. It is worth noting that the difference between the results of the two algorithms is very obvious, which indicates that the hybrid initialization strategy in this paper has a great effect on improving NSGA-II. Similarly, the results of algorithms B and C illustrate that the neighborhood search strategy in this paper is necessary. Finally, the results of QNSGA-II are better than that of algorithm C on most instances, which verifies the effectiveness of Q -Learning algorithm. Table 6 shows that a large fraction of the solutions obtained by QNSGA-II can dominate the solutions produced by other algorithms.

6.4. Comparative analysis with other algorithms

In this section, QNSGA-II is compared with three algorithms to verify the performance of the proposed algorithm. The three algorithms are NSGA-II, MOEA/D and MOPSO. Set the public parameter $N = 100$, the CPU running time of the three comparison algorithms is the same as QNSGA-II. The respective parameters of the algorithms are shown in Table 9. Each algorithm is run 20 times on each instance, and the average HV, IGD and C-metric values obtained by each algorithm for each instance are presented in Table 7 and Table 8. Furthermore, three different scale instances EMK01, EMK04 and EMK05 are selected, and the boxplots of the two objective values solved by the four algorithms are depicted. From Fig. 8, it can be observed that QNSGA-II performs the best in minimizing makespan and TEC. The test results above demonstrate the superiority of the QNSGA-II algorithm compared to others.

6.5. Effectiveness of the rescheduling strategies

In order to evaluate the performance of the three rescheduling strategies, including their performance in minimizing the two objective values of makespan and TEC, and the stability of the generated rescheduling solution, we use (1) the strategy of executing the original scheduling scheme IS, (2) the rescheduling strategy of this paper RS, (3) the complete rescheduling strategy CS for strategy comparison. In order to simulate the demand for timely response of rescheduling solution in flexible job shop in real production environment, this paper sets the running time of rescheduling to be half of the running time of the initial scheduling, and the CPU running time of the three rescheduling strategies on the same instance is equal. Examples of different scales are selected for each of the three disturbance events. In each instance, the disturbances generated in different periods are analyzed, and the impact of maintenance time after machine breakdown and AGV breakdown on the rescheduling solution is studied.

$$\bar{\tau}_{i,j} = \frac{\tau_{i,j} - \min \tau_i}{\max \tau_i - \min \tau_i} \quad (22)$$

For example, EMK01-20-5 in Table 11 represents a breakdown occurred at time 10 for a machine in instance EMK02 with a repair time of 5 units. Each instance is independently run 20 times to calculate average values for two objectives: makespan and TEC from 10 solutions obtained. The average values for three sub-indicators mentioned in 4.2: MCR, σ_a , APSD are also calculated. The solutions generated by all three algorithms are combined and normalized using Eq. (22), with weights $\lambda_1, \lambda_2, \lambda_3$ set as 0.3333, 0.3333, 0.3333. RSI is then calculated according to Eq. (17) and averaged across all three algorithms' RSI values. Detailed results for job cancellation, machine breakdown, and AGV breakdown are shown in Table 10, Table 11, and Table 12 respectively. The tables show the objective values obtained by three rescheduling strategies, including makespan and TEC. Additionally, the tables display their performance in terms of three rescheduling-related sub-indicators and the RSI.

Table 10 shows the results under the disturbance event of job cancellation. We can find that the solution obtained by RS_1 is better

Table 5

Result of the HV and IGD obtained by QNSGA-II, A, B and C on each instance.

Instances	QNSGA-II		A		B		C	
	HV	IGD	HV	IGD	HV	IGD	HV	IGD
EMK01	1.1319	0.0937	0.0320	1.1968	0.8918	0.1919	1.0380	0.1474
EMK02	1.1194	0.0384	0.0379	1.1998	0.8036	0.2552	1.0648	0.1142
EMK03	1.0923	0.0600	0.0204	1.2881	0.6646	0.3885	0.9317	0.2371
EMK04	1.0789	0.0655	0.0158	1.3000	0.9864	0.1459	0.9895	0.1568
EMK05	1.0806	0.0623	0.0294	1.2126	0.9849	0.1818	1.0251	0.0813
EMK06	1.0604	0.0683	0.0214	1.2371	0.6465	0.4552	0.7958	0.3134
EMK07	1.1466	0.0347	0.0286	1.1940	0.9151	0.1478	0.9999	0.1530
EMK08	1.0707	0.1078	0.0252	1.3060	0.8085	0.2898	0.9462	0.2067
EMK09	1.0148	0.0979	0.0225	1.2098	0.6693	0.3742	0.8914	0.2252
EMK10	1.1103	0.0603	0.0171	1.3187	0.5728	0.4713	1.0577	0.1081

Table 6

Result of the C-metric on by QNSGA-II, A, B and C each instance.

Instances	C(QNSGA-II,A)	C(A,QNSGA-II)	C(QNSGA-II,B)	C(B,QNSGA-II)	C(QNSGA-II,C)	C(C,QNSGA-II)
EMK01	1.0000	0.0000	0.8000	0.1000	0.6933	0.1333
EMK02	1.0000	0.0000	0.9000	0.1000	0.6000	0.1000
EMK03	1.0000	0.0000	1.0000	0.0000	0.6000	0.1817
EMK04	1.0000	0.0000	0.5714	0.3476	0.5714	0.3333
EMK05	1.0000	0.0000	0.5600	0.3400	0.4167	0.3500
EMK06	1.0000	0.0000	0.7778	0.1154	0.5741	0.0598
EMK07	1.0000	0.0000	0.6833	0.1917	0.7946	0.0000
EMK08	1.0000	0.0000	0.8750	0.1250	0.4063	0.5000
EMK09	1.0000	0.0000	1.0000	0.0000	0.3167	0.1026
EMK10	1.0000	0.0000	1.0000	0.0000	0.4444	0.4167

Table 7

Result of the HV and IGD obtained by multiple algorithms on each instance.

Instances	QNSGA-II		NSGA-II		MOEA/D		MOPSO	
	HV	IGD	HV	IGD	HV	IGD	HV	IGD
EMK01	1.1954	0.0000	0.3909	0.6508	0.3100	0.7310	0.0567	1.1709
EMK02	1.1980	0.0000	0.4422	0.6270	0.4629	0.5816	0.0456	1.2503
EMK03	1.1996	0.0000	0.3540	0.7313	0.3754	0.6880	0.0560	1.2160
EMK04	1.2079	0.0000	0.3949	0.6588	0.3879	0.6510	0.0577	1.2116
EMK05	1.2076	0.0000	0.2776	0.7835	0.2139	0.8614	0.0298	1.2555
EMK06	1.2094	0.0000	0.4368	0.6412	0.3787	0.6947	0.0366	1.2889
EMK07	1.2099	0.0000	0.4354	0.6471	0.4557	0.6085	0.1021	1.1342
EMK08	1.2098	0.0000	0.3964	0.6966	0.3344	0.7477	0.0302	1.2020
EMK09	1.1969	0.0000	0.5895	0.4982	0.6604	0.4298	0.0584	1.2228
EMK10	1.1989	0.0000	0.4781	0.5812	0.3741	0.6841	0.0413	1.2696

Table 8

Result of the C-metric obtained by multiple algorithms on each instance.

Instances	C(QNSGA-II, NSGA-II)	C(NSGA-II, QNSGA-II)	C(QNSGA-II, MOPSO)	C(MOPSO, QNSGA-II)	C(QNSGA-II, MOEA/D)	C(NSGA-II, QNSGA-II)
EMK01	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000
EMK02	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000
EMK03	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000
EMK04	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000
EMK05	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000
EMK06	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000
EMK07	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000
EMK08	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000
EMK09	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000
EMK10	1.0000	0.0000	1.0000	0.0000	1.0000	0.0000

Table 9

Parameter setting.

Algorithm	Parameters
QNSGA-II	$P_c=0.8$, $P_m=0.1$, $\alpha=0.1$, $\gamma=0.9$
NSGA-II	$P_c=0.8$, $P_m=0.1$
MOEA/D	$T=5$
MOPSO	$W=0.4$, $C_1=2$, $C_2=2$

than the IS solution in all instances. In the disturbance event of job cancellation, the RS_1 strategy and the IS strategy optimize the makespan and TEC by shifting the subsequent operations of the cancellation

job to the left, using the idle time of the machine and AGV. The difference between the RS_1 solution and the IS solution is that RS_1 can reschedule the AGV to advance the start processing time of the job to a greater extent at the expense of the start processing time deviation of a small number of jobs.

Compared with the CS solution, the RS_1 solution also performs better in most instances. This is because RS_1 maintains the machine sequence of each operation and the sequential processing order of jobs on all machines, maintains the stability of the OS and MS coding segments, and reduces the running complexity of the algorithm. In limited solution time, RS_1 can converge to a satisfactory solution faster. For the CS strategy, it is better than the RS_1 solution only

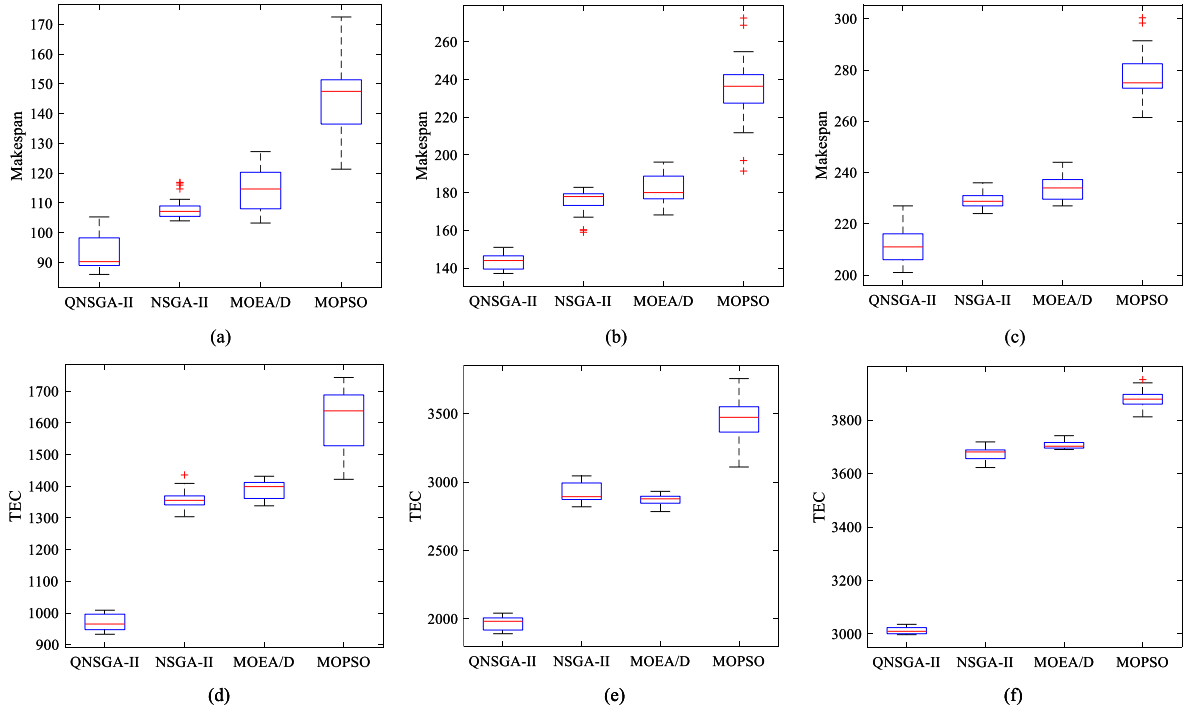


Fig. 8. Boxplots of the four algorithms solving for the objective values.

Table 10

Rescheduling results of three strategies under job cancellation event.

Instances	IS				RS-3				CS			
	Makespan		TEC		Makespan		TEC		Makespan		TEC	
EMK02-20	68.5000		924.0600		67.0000		910.5500		61.0000		879.3000	
EMK02-50	72.6000		946.6333		67.6667		913.3000		69.4161		933.4750	
EMK07-50	173.8633		3097.7823		173.0000		3088.4333		173.3333		3116.9153	
EMK07-100	181.0000		3154.0153		174.6667		3144.3333		180.6125		3158.8908	
Instances	Sub-indicators			RSI	Sub-indicators			RSI	Sub-indicators			RSI
	MCR	σ_a	APSD		MCR	σ_a	APSD		MCR	σ_a	APSD	
EMK02-20	0.0000	3.2413	1.0000	0.2409	0.0000	3.2381	1.2800	0.2412	0.1667	2.8575	2.0386	0.4218
EMK02-50	0.0000	3.3246	0.0000	0.3452	0.0000	3.1687	1.1333	0.3878	0.6667	2.2378	5.0475	0.7233
EMK07-50	0.0000	4.8235	1.3533	0.2361	0.0000	4.7147	1.4500	0.1874	0.5113	4.9863	4.2271	0.8069
EMK07-100	0.0000	4.6197	0.6667	0.1021	0.0000	4.6476	0.7518	0.1156	0.4051	5.6330	4.0897	0.8824

Table 11

Rescheduling results of three strategies under machine breakdown event.

Instances	IS				RS-3				CS			
	Makespan		TEC		Makespan		TEC		Makespan		TEC	
EMK01-20-5	98.0000		1027.2000		98.5600		973.0160		112.3200		1276.0040	
EMK01-20-10	104.4100		1084.9017		99.6667		1057.3707		117.2800		1195.6080	
EMK01-50-5	95.3100		1093.8417		94.5933		1069.1813		105.7933		1166.4920	
EMK01-50-10	99.3133		1118.5597		97.7633		1108.1387		112.0100		1193.3600	
EMK06-100-15	328.0367		5134.4385		326.6800		5185.8900		354.5250		5268.4910	
EMK06-100-25	336.2950		5273.7233		328.9633		5185.4540		359.7900		5566.0987	
EMK06-200-15	337.8600		5826.9648		324.7800		5758.5085		350.8650		5905.6420	
EMK06-200-25	335.0186		5943.9803		330.5129		5717.1657		370.5000		6317.6500	
Instances	Sub-indicators			RSI	Sub-indicators			RSI	Sub-indicators			RSI
	MCR	σ_a	APSD		MCR	σ_a	APSD		MCR	σ_a	APSD	
EMK01-20-5	0.0000	4.6798	1.1323	0.0620	0.0089	5.3600	2.7333	0.2265	0.3637	6.0233	37.1565	0.9412
EMK01-20-10	0.0000	4.2708	3.4829	0.1322	0.0554	4.2987	4.9303	0.2204	0.3024	4.9370	22.4174	0.9028
EMK01-50-5	0.0000	4.2335	1.0817	0.1847	0.0945	4.0718	2.9739	0.3245	0.4103	3.9385	7.0952	0.6611
EMK01-50-10	0.0000	3.7295	3.0498	0.0902	0.0476	3.7759	4.1460	0.2497	0.2049	5.0606	25.7684	0.9021
EMK06-100-15	0.0000	5.8984	5.8845	0.2073	0.1057	5.8459	6.9594	0.3039	0.5409	6.5227	35.9147	0.8016
EMK06-100-25	0.0000	6.0978	10.9820	0.2241	0.1789	6.0616	19.0565	0.3330	0.4785	6.5162	26.9355	0.7183
EMK06-200-15	0.0000	5.8218	5.6706	0.2236	0.0783	5.7249	11.0223	0.2767	0.5749	6.2887	39.6907	0.7826
EMK06-200-25	0.0000	5.6031	7.5661	0.1099	0.0920	5.7141	10.7027	0.1823	0.6549	8.6600	31.1610	0.7205

Table 12
Rescheduling results of three strategies under AGV breakdown event.

Instances	IS		RS-3		CS	
	Makespan	TEC	Makespan	TEC	Makespan	TEC
EMK04-20-5	148.3100	2114.7778	146.1250	2049.9528	157.3260	2358.6018
EMK04-20-10	153.0550	2088.6893	151.0775	2095.4343	162.8540	2449.1048
EMK04-50-5	140.1960	2057.6994	137.9575	2053.0273	149.0940	2252.3120
EMK04-50-10	156.6140	2149.6112	144.3800	2102.8510	148.3880	2293.5832
EMK10-200-15	475.5750	14134.0370	461.9900	13583.7320	505.9775	14922.8278
EMK10-200-25	473.0567	13745.3540	464.4133	13044.8563	507.4400	14888.2853
EMK10-300-15	464.2450	14168.0208	453.0933	14003.3750	480.9275	14684.8393
EMK10-300-25	473.8840	14215.8230	460.7500	14095.9243	468.7580	14537.8930

Instances	Sub-indicators			RSI	Sub-indicators			RSI	Sub-indicators			RSI
	MCR	σ_a	APSD		MCR	σ_a	APSD		MCR	σ_a	APSD	
EMK04-20-5	0.0000	5.5158	7.8220	0.1966	0.0000	5.7033	9.6450	0.2691	0.2579	5.9589	29.5960	0.7852
EMK04-20-10	0.0000	5.6930	11.9950	0.1853	0.0000	5.9219	13.9350	0.2687	0.2972	6.1601	21.8640	0.7221
EMK04-50-5	0.0000	5.0160	4.9360	0.1588	0.0000	5.1199	9.1550	0.2253	0.2595	5.4789	17.3960	0.6466
EMK04-50-10	0.0000	5.2354	14.8640	0.2062	0.0000	5.1818	11.5733	0.1637	0.2657	5.4399	18.5720	0.5856
EMK10-200-15	0.0000	6.5146	2.7625	0.0911	0.0000	6.4368	14.8200	0.1490	0.5764	7.0042	37.5425	0.7683
EMK10-200-25	0.0000	6.3175	24.7167	0.1376	0.0000	6.3428	25.5567	0.1463	0.5136	7.1298	75.5000	0.8352
EMK10-300-15	0.0000	7.2898	9.7475	0.2029	0.0000	6.7194	9.7933	0.1576	0.5292	6.9116	22.4125	0.6279
EMK10-300-25	0.0000	6.1827	18.1900	0.2612	0.0000	6.2469	14.4050	0.2548	0.4980	6.4224	27.3440	0.7195

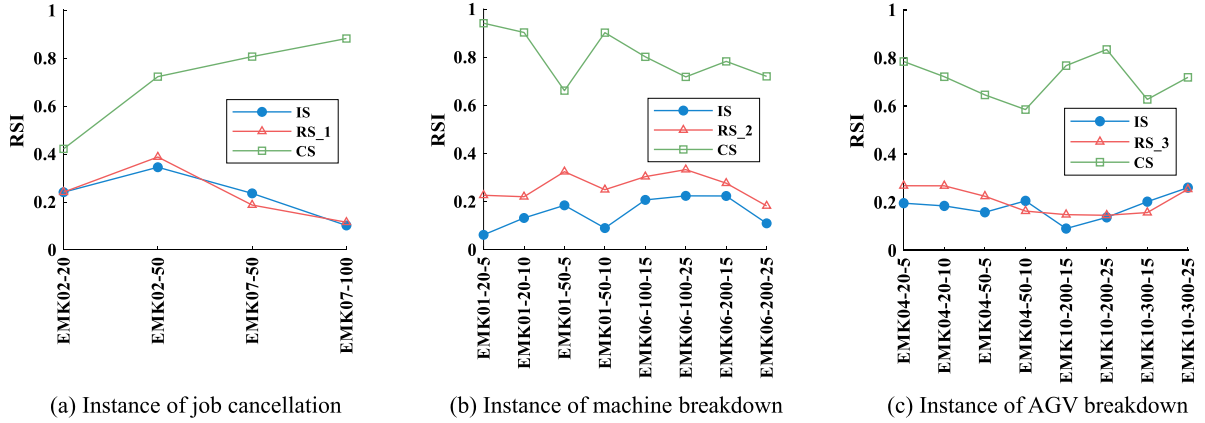


Fig. 9. RSI of different strategies under three disturbance events.

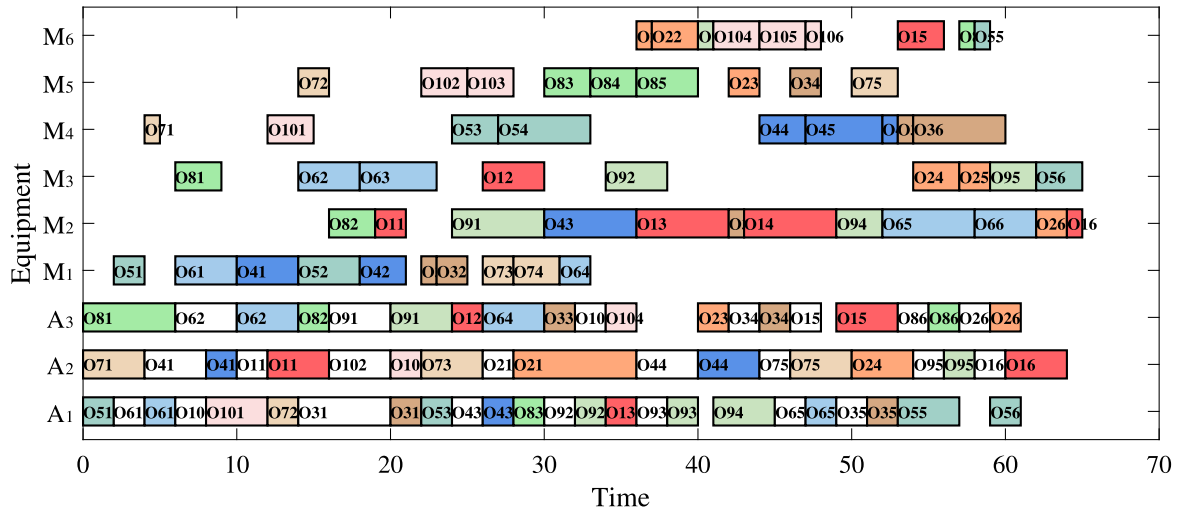


Fig. 10. Gantt chart of EMK02 instance (Makespan = 65, TEC = 960.5).

on the small-scale instance EMK02-20. As the instance size increases, it becomes increasingly difficult for CS to obtain a solution of high quality. At the same time, the CS strategy cannot guarantee the stability of the rescheduling solution, which is reflected in its poor performance on RSI.

Table 11 shows the results of three rescheduling solutions under machine breakdown. Similarly, since the RS_2 strategy ensures that jobs on non-broken machines are still processed on the original machine, RS_2 can still obtain a more satisfactory solution in a limited number of iterations, and also maintains good stability. Under different

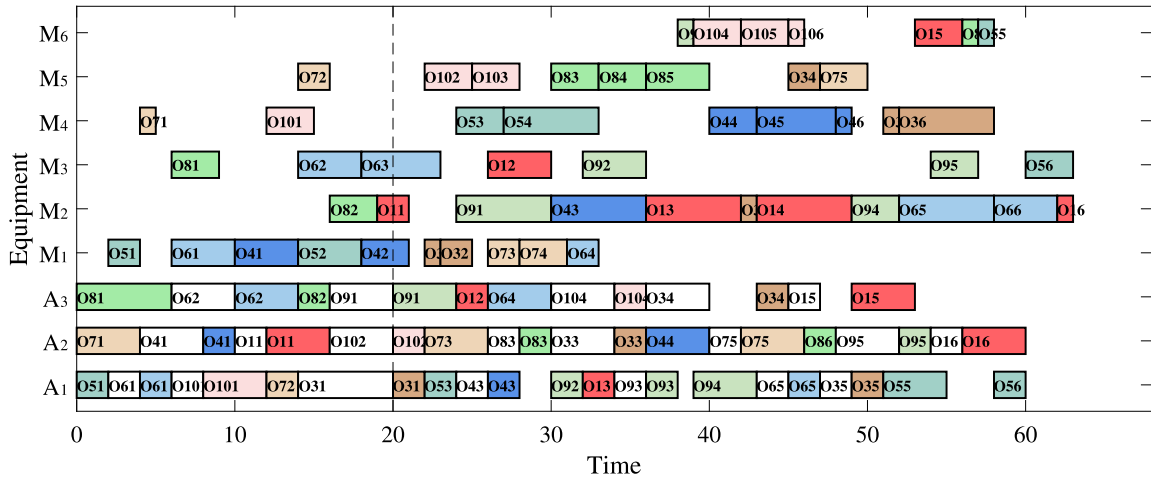


Fig. 11. Gantt chart of rescheduling solution for RS_1 strategy(Makespan = 63, TEC = 897.8).

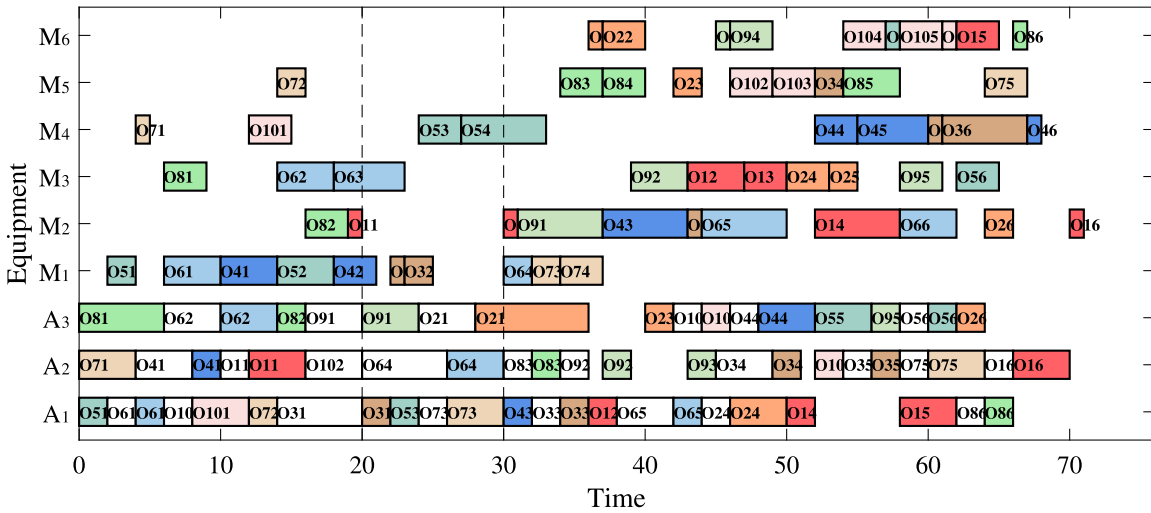


Fig. 12. Gantt chart of rescheduling solution for RS_2 strategy(Makespan = 71, TEC = 986.9).

breakdown occurrence times and different maintenance durations, the solutions obtained by the RS_2 strategy are better than the IS solution and the CS solution in most instances. In instances EMK01-20-5 and EMK06-100-15, the RS_2 solution does not dominate the IS solution, probably because the disturbance caused by machine breakdown is more likely to be absorbed by the subsequent scheduling process.

Table 12 shows the comparison results under the disturbance event of AGV breakdown. Similarly, we can find that RS_3 is better than the other two solutions in both targets of makespan and TEC. Compared with the IS strategy, the RS_3 strategy reduces the constraints on the job processing sequence and AGV scheduling. In this way, the processing time of other jobs on the machine can be reasonably advanced and the delayed arrival time of jobs due to AGV breakdown can be reduced. In addition, we found that the RSI value of the RS_3 solution is better than that of the IS solution on the EMK04-50-10, EMK10-300-15, and EMK10-300-25 instances. This may indicate that under the condition of tight transportation resources, if the AGV breakdown occurs for a long time in the later scheduling period, the disturbance caused by the breakdown cannot be eliminated well by continuing to execute the initial solution.

Fig. 9 illustrates the RSI of different strategies under three types of disturbance events. It can be observed that the RSI of the proposed RS strategy is comparable to that of the IS strategy. Particularly in scenarios involving job cancellation and AGV breakdown, the RS strategy even outperforms the IS strategy in some instances, demonstrating its

effectiveness in maintaining rescheduling stability. Furthermore, when comparing the RSI across the three disturbance events, it can be found that the machine breakdown is the most disturbing to the scheduling solution. Additionally, the RSI of the CS strategy is significantly higher than that of the other two strategies, indicating a substantial deviation from the initial scheduling solution, which undermines the stability of the rescheduling solution.

To sum up, this paper designs three different rescheduling strategies RS based on the characteristics of different disturbance events, which can simplify the algorithm complexity to obtain better performance solutions under the condition of limited scheduling time, while taking into account the stability of rescheduling solutions. Due to the limited rescheduling time, the algorithm of complete rescheduling cannot converge to the optimal solution in the specified time. Comparing the RS solution with the CS solution, it can be found that the complete rescheduling solution is inferior to the RS solution in terms of the values on the two objectives as well as the RSI. It follows that complete rescheduling is not an appropriate strategy in time-limited rescheduling problems.

Fig. 10 shows the gantt chart of the initial scheduling solution for the EMK02 instance. The occurrence time of the disturbance events is set at time 10, and the maintenance time for machine and AGV breakdown is set to 10. The gantt charts of the rescheduling solution under the influence of three disturbance events such as job cancellation, machine breakdown, and AGV breakdown are shown in Fig. 11, 12,

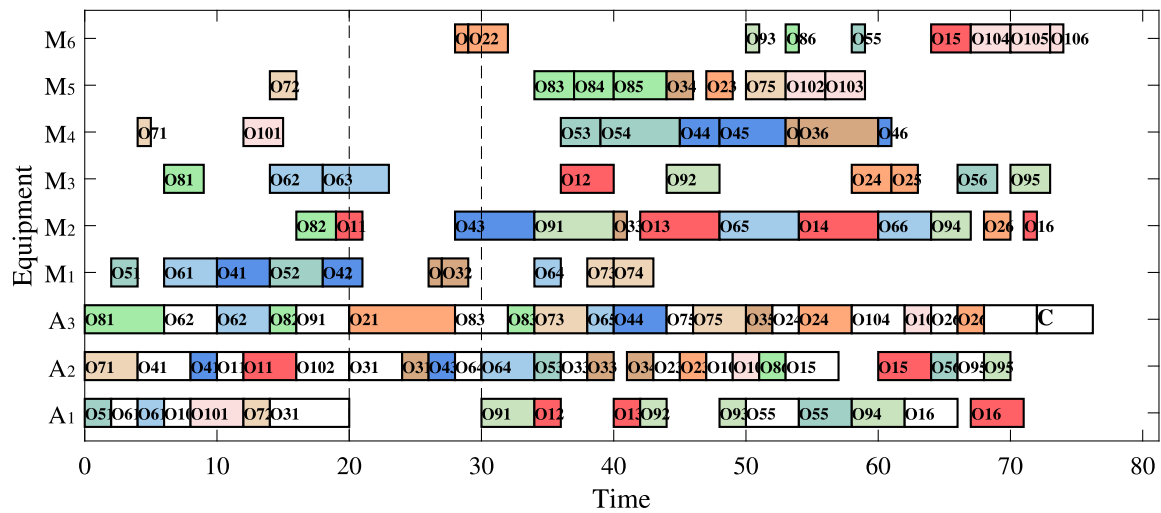


Fig. 13. Gantt chart of rescheduling solution for RS_3 strategy (Makespan = 74, TEC = 1004).

and 13. Makespan and TEC are also calculated for each rescheduling solution.

7. Conclusions

This paper proposes a dynamic scheduling problem for the flexible job shop with limited transportation resources. A rescheduling model is constructed to minimize makespan and TEC; corresponding rescheduling strategies under three disturbance events such as job cancellation, machine breakdown and AGV breakdown are proposed, and a rescheduling instability indicator is introduced. Compared with the strategy of continuing to execute the initial scheduling solution and the complete rescheduling strategy, the rescheduling strategies RS can obtain the best solution at the two target values. At the same time, it also performs well in terms of the stability of the rescheduling solution. Furthermore, a *Q*-Learning based QNSGA-II is proposed to solve this problem. By combining *Q*-Learning and local search, historical search experience is used to guide the selection of neighborhood structures. And a problem-oriented mixed population strategy is proposed, which effectively improves the quality and distribution of the initial population. A large number of comparative experiments show that QNSGA-II can effectively solve DFJSP-T.

In future work, we will continue to study other disturbance events that may occur in DFJSP-T. At the same time, we will also explore how to apply deep reinforcement learning to solve this problem. In addition, how to design an efficient neighborhood structure for the FJSP-T problem is also a focus of future research.

CRedit authorship contribution statement

Rensheng Chen: Writing – original draft, Methodology, Conceptualization. **Bin Wu:** Writing – review & editing, Supervision, Conceptualization. **Hua Wang:** Writing – review & editing, Project administration. **Huagang Tong:** Visualization, Validation. **Feiyi Yan:** Software.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

This work is supported by the General Project of the National Social Science Fund of China (Grant No. 20BGL025), the National Key Research and Development Program of China (Grant No. 2021YFB3301300).

References

- [1] P.L. Mareddy, S.R. Narapureddy, V.R. Dwivedula, P.R. Karanam, Development of scheduling methodology in a multi-machine flexible manufacturing system without tool delay employing flower pollination algorithm, *Eng. Appl. Artif. Intell.* 115 (2022) 105275, <http://dx.doi.org/10.1016/j.engappai.2022.105275>.
- [2] J. Xie, L. Gao, K. Peng, X. Li, H. Li, Review on flexible job shop scheduling, *IET Collaborat. Intell. Manuf.* 1 (3) (2019) 67–77.
- [3] M. Mousavi, H.J. Yap, S.N. Musa, F. Tahriri, S.Z. Md Dawal, Multi-objective AGV scheduling in an FMS using a hybrid of genetic algorithm and particle swarm optimization, *PLoS One* 12 (3) (2017) e0169817.
- [4] B. Reddy, C. Rao, A hybrid multi-objective GA for simultaneous scheduling of machines and AGVs in FMS, *Int. J. Adv. Manuf. Technol.* 31 (5) (2006) 602–613.
- [5] I.A. Chaudhry, S. Mahmood, M. Shami, Simultaneous scheduling of machines and automated guided vehicles in flexible manufacturing systems using genetic algorithms, *J. Cent. S. Univ.* 18 (2011) 1473–1486.
- [6] S.M. Homayouni, D.B. Fontes, Production and transport scheduling in flexible job shop manufacturing systems, *J. Global Optim.* 79 (2) (2021) 463–502.
- [7] Z. Wang, J. Zhang, S. Yang, An improved particle swarm optimization algorithm for dynamic job shop scheduling problems with random job arrivals, *Swarm Evol. Comput.* 51 (2019) 100594, <http://dx.doi.org/10.1016/j.swevo.2019.100594>.
- [8] J. Duan, J. Wang, Robust scheduling for flexible machining job shop subject to machine breakdowns and new job arrivals considering system reusability and task recurrence, *Expert Syst. Appl.* 203 (2022) 117489, <http://dx.doi.org/10.1016/j.eswa.2022.117489>.
- [9] W. Ren, Y. Yan, Y. Hu, Y. Guan, Joint optimisation for dynamic flexible job-shop scheduling problem with transportation time and resource constraints, *Int. J. Prod. Res.* 60 (18) (2022) 5675–5696.
- [10] J. Liu, B. Sun, G. Li, Y. Chen, Multi-objective adaptive large neighbourhood search algorithm for dynamic flexible job shop schedule problem with transportation resource, *Eng. Appl. Artif. Intell.* 132 (2024) 107917, <http://dx.doi.org/10.1016/j.engappai.2024.107917>.
- [11] I. Driss, K.N. Mouss, A. Laggoun, A new genetic algorithm for flexible job-shop scheduling problems, *J. Mech. Sci. Technol.* 29 (2015) 1273–1281.
- [12] P. Ngatchou, A. Zarei, A. El-Sharkawi, Pareto multi objective optimization, in: *Proceedings of the 13th International Conference on Intelligent Systems Application To Power Systems*, IEEE, 2005, pp. 84–91.
- [13] K. Gao, Z. Cao, L. Zhang, Z. Chen, Y. Han, Q. Pan, A review on swarm intelligence and evolutionary algorithms for solving flexible job shop scheduling problems, *IEEE/CAA J. Autom. Sin.* 6 (4) (2019) 904–916.
- [14] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Trans. Evol. Comput.* 6 (2) (2002) 182–197.
- [15] M.E.H. Sadati, B. Çatay, A hybrid variable neighborhood search approach for the multi-depot green vehicle routing problem, *Transp. Res. Part E: Logist. Transp. Rev.* 149 (2021) 102293, <http://dx.doi.org/10.1016/j.tre.2021.102293>.

- [16] Z. Pan, L. Wang, J. Wang, J. Lu, Deep reinforcement learning based optimization algorithm for permutation flow-shop scheduling, *IEEE Trans. Emerg. Top. Comput. Intell.* 7 (4) (2021) 983–994.
- [17] L. Deroussi, S. Norre, Simultaneous scheduling of machines and vehicles for the flexible job shop problem, in: *International Conference on Metaheuristics and Nature Inspired Computing*, Djerba Island Tunisia, 2010, pp. 1–2.
- [18] Z. Pan, L. Wang, J. Zheng, J.-F. Chen, X. Wang, A learning-based multi-population evolutionary optimization for flexible job shop scheduling problem with finite transportation resources, *IEEE Trans. Evol. Comput.* (2022).
- [19] J. Liu, B. Sun, G. Li, Y. Chen, An integrated scheduling approach considering dispatching strategy and conflict-free route of AMRs in flexible job shop, *Int. J. Adv. Manuf. Technol.* 127 (3) (2023) 1979–2002.
- [20] J. Yan, Z. Liu, C. Zhang, T. Zhang, Y. Zhang, C. Yang, Research on flexible job shop scheduling under finite transportation conditions for digital twin workshop, *Robot. Comput.-Integr. Manuf.* 72 (2021) 102198.
- [21] G. Xu, Q. Bao, H. Zhang, Multi-objective green scheduling of integrated flexible job shop and automated guided vehicles, *Eng. Appl. Artif. Intell.* 126 (2023) 106864.
- [22] J. Li, J. Deng, C. Li, Y. Han, J. Tian, B. Zhang, C. Wang, An improved Jaya algorithm for solving the flexible job shop scheduling problem with transportation and setup times, *Knowl.-Based Syst.* 200 (2020) 106032, <http://dx.doi.org/10.1016/j.knsys.2020.106032>.
- [23] X. Lyu, Y. Song, C. He, Q. Lei, W. Guo, Approach to integrated scheduling problems considering optimal number of automated guided vehicles and conflict-free routing in flexible manufacturing systems, *IEEE Access* 7 (2019) 74909–74924.
- [24] M.S. Kumar, R. Janardhana, C. Rao, Simultaneous scheduling of machines and vehicles in an FMS environment with alternative routing, *Int. J. Adv. Manuf. Technol.* 53 (2011) 339–351.
- [25] H.E. Nouri, O. Belkahlia Driss, K. Ghédira, Simultaneous scheduling of machines and transport robots in flexible job shop environment using hybrid metaheuristics based on clustered holonic multiagent model, *Comput. Ind. Eng.* 102 (2016) 488–501, <http://dx.doi.org/10.1016/j.cie.2016.02.024>.
- [26] S. Luo, Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning, *Appl. Soft Comput.* 91 (2020) 106208, <http://dx.doi.org/10.1016/j.asoc.2020.106208>.
- [27] L. Cai, W. Li, Y. Luo, L. He, Real-time scheduling simulation optimisation of job shop in a production-logistics collaborative environment, *Int. J. Prod. Res.* 61 (5) (2023) 1373–1393.
- [28] M. Ghaleb, H. Zolfaghariania, S. Taghipour, Real-time production scheduling in the Industry-4.0 context: Addressing uncertainties in job arrivals and machine breakdowns, *Comput. Oper. Res.* 123 (2020) 105031, <http://dx.doi.org/10.1016/j.cor.2020.105031>.
- [29] H. Luo, J. Fang, G.Q. Huang, Real-time scheduling for hybrid flowshop in ubiquitous manufacturing environment, *Comput. Ind. Eng.* 84 (2015) 12–23.
- [30] Y. Li, W. Gu, M. Yuan, Y. Tang, Real-time data-driven dynamic scheduling for flexible job shop with insufficient transportation resources using hybrid deep q network, *Robot. Comput.-Integr. Manuf.* 74 (2022) 102283.
- [31] L. Liu, H.-y. Gu, Y.-g. Xi, Robust and stable scheduling of a single machine with random machine breakdowns, *Int. J. Adv. Manuf. Technol.* 31 (2007) 645–654.
- [32] Y. Yang, M. Huang, Z. Yu Wang, Q. Bing Zhu, Robust scheduling based on extreme learning machine for bi-objective flexible job-shop problems with machine breakdowns, *Expert Syst. Appl.* 158 (2020) 113545, <http://dx.doi.org/10.1016/j.eswa.2020.113545>.
- [33] N. Zhu, G. Gong, D. Lu, D. Huang, N. Peng, H. Qi, An effective reformative memetic algorithm for distributed flexible job-shop scheduling problem with order cancellation, *Expert Syst. Appl.* 237 (2024) 121205.
- [34] X. long Chen, J. qing Li, Y. Xu, Q-learning based multi-objective immune algorithm for fuzzy flexible job shop scheduling problem considering dynamic disruptions, *Swarm Evol. Comput.* 83 (2023) 101414, <http://dx.doi.org/10.1016/j.swevo.2023.101414>.
- [35] M. Gholami, M. Zandieh, Integrating simulation and genetic algorithm to schedule a dynamic flexible job shop, *J. Intell. Manuf.* 20 (2009) 481–498.
- [36] M. Adibi, M. Zandieh, M. Amiri, Multi-objective scheduling of dynamic job shop using variable neighborhood search, *Expert Syst. Appl.* 37 (1) (2010) 282–287, <http://dx.doi.org/10.1016/j.eswa.2009.05.001>, URL: <https://www.sciencedirect.com/science/article/pii/S0957417409004199>.
- [37] X. Li, Z. Peng, B. Du, J. Guo, W. Xu, K. Zhuang, Hybrid artificial bee colony algorithm with a rescheduling strategy for solving flexible job shop scheduling problems, *Comput. Ind. Eng.* 113 (2017) 10–26.
- [38] R. Chen, B. Yang, S. Li, S. Wang, A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem, *Comput. Ind. Eng.* 149 (2020) 106778, <http://dx.doi.org/10.1016/j.cie.2020.106778>.
- [39] J. Shahrahi, M.A. Adibi, M. Mahootchi, A reinforcement learning approach to parameter estimation in dynamic job shop scheduling, *Comput. Ind. Eng.* 110 (2017) 75–82, <http://dx.doi.org/10.1016/j.cie.2017.05.026>, URL: <https://www.sciencedirect.com/science/article/pii/S0360835217302309>.
- [40] Y. Yao, X. Li, L. Gao, A DQN-based memetic algorithm for energy-efficient job shop scheduling problem with integrated limited AGVs, *Swarm Evol. Comput.* 87 (2024) 101544.
- [41] F. Zhang, R. Li, W. Gong, Deep reinforcement learning-based memetic algorithm for energy-aware flexible job shop scheduling with multi-AGV, *Comput. Ind. Eng.* 189 (2024) 109917.
- [42] F. Zhao, S. Di, L. Wang, A hyperheuristic with Q-learning for the multiobjective energy-efficient distributed blocking flow shop scheduling problem, *IEEE Trans. Cybern.* (2022).
- [43] H. Li, K. Gao, P.-Y. Duan, J.-Q. Li, L. Zhang, An improved artificial bee colony algorithm with Q-learning for solving permutation flow-shop scheduling problems, *IEEE Trans. Syst., Man, Cybern. Syst.* 53 (5) (2022) 2684–2693.
- [44] J. Wang, D. Lei, J. Cai, An adaptive artificial bee colony with reinforcement learning for distributed three-stage assembly scheduling with maintenance, *Appl. Soft Comput.* 117 (2022) 108371, <http://dx.doi.org/10.1016/j.asoc.2021.108371>.
- [45] Y.-F. Wang, Adaptive job shop scheduling strategy based on weighted Q-learning algorithm, *J. Intell. Manuf.* 31 (2) (2020) 417–432.
- [46] L. Cheng, Q. Tang, L. Zhang, Z. Zhang, Multi-objective Q-learning-based hyperheuristic with bi-criteria selection for energy-aware mixed shop scheduling, *Swarm Evol. Comput.* 69 (2022) 100985, <http://dx.doi.org/10.1016/j.swevo.2021.100985>.
- [47] Y. Li, M. Han, Q. Guo, Modified whale optimization algorithm based on tent chaotic mapping and its application in structural optimization, *KSCE J. Civ. Eng.* 24 (12) (2020) 3703–3713.
- [48] R. Li, W. Gong, L. Wang, C. Lu, S. Jiang, Two-stage knowledge-driven evolutionary algorithm for distributed green flexible job shop scheduling with type-2 fuzzy processing time, *Swarm Evol. Comput.* 74 (2022) 101139.
- [49] Y. An, X. Chen, K. Gao, L. Zhang, Y. Li, Z. Zhao, Integrated optimization of real-time order acceptance and flexible job-shop rescheduling with multi-level imperfect maintenance constraints, *Swarm Evol. Comput.* 77 (2023) 101243.
- [50] P. Brandimarte, Routing and scheduling in a flexible job shop by tabu search, *Ann. Oper. Res.* 41 (3) (1993) 157–183.
- [51] A.P. Guerreiro, C.M. Fonseca, L. Paquete, The hypervolume indicator: Computational problems and algorithms, *ACM Comput. Surv.* 54 (6) (2021) 1–42.
- [52] H. Ishibuchi, H. Masuda, Y. Tanigaki, Y. Nojima, Modified distance calculation in generational distance and inverted generational distance, in: *Evolutionary Multi-Criterion Optimization: 8th International Conference, EMO 2015, Guimarães, Portugal, March 29–April 1, 2015. Proceedings, Part II* 8, Springer, 2015, pp. 110–125.
- [53] N. Riquelme, C. Von Lücken, B. Baran, Performance metrics in multi-objective optimization, in: *2015 Latin American Computing Conference, CLEI, IEEE, 2015*, pp. 1–11.